

The proofgraph package

Pierre Senellart
pierre@senellart.com

2026/07/27 v1.0.1-dev

Abstract

The `proofgraph` package automatically produces a graph of the dependencies between the results (theorems, lemmas, propositions) of a mathematical article. It infers an edge from one result to another whenever the proof of the former refers to the latter through an ordinary cross-reference, so that most dependencies need no manual annotation; commands are provided for those a visible reference does not capture. The graph is written as a `Graphviz .dot` file, which can be rendered externally or, optionally, embedded into the document automatically.

1 Introduction

When writing or reading a mathematical article, it is often useful to visualise how the results depend on one another: which lemma is used in the proof of which theorem, and so on. `proofgraph` builds such a dependency graph automatically. Each theorem-like environment becomes a node; each cross-reference (`\ref`, `\cref`...) appearing *inside a proof* becomes an edge from the proved result to the referred-to result. Usually no manual annotation is required, and commands are provided for the dependencies a visible reference does not capture.

The package is best explained by a small example of its use. This manual is itself a `proofgraph` document, so the results stated below are typeset with the package enabled, and the graph shown at the end of this section is the one they produce.

Load `proofgraph` in the preamble, before the `\newtheorem` commands that declare your theorem-like environments (so it sees them as they are created). Here, as in this manual, we pass three options: `autorun` renders the graph with `Graphviz` and embeds it automatically (it needs shell-escape); `cite` captures the `\cites` made inside proofs as dependencies on external work; and `citelabel=tag` labels those citation nodes by their printed tag:

```
\usepackage[autorun,cite,citelabel=tag]{proofgraph}
\newtheorem{theorem}{Theorem}
\newtheorem{lemma}{Lemma}
\newtheorem{corollary}{Corollary}
```

Then state and prove, as usual, a few results that build on one another:

```
\begin{lemma}\label{lem:even}
  The sum of two even integers is even.
\end{lemma}
```

```

\begin{lemma}\label{lem:odd}
  The sum of two odd integers is even.
\end{lemma}
\begin{theorem}\label{thm:parity}
  The sum of two integers of the same parity is even.
\end{theorem}
\begin{proof}
  By Lemmas~\ref{lem:even} and~\ref{lem:odd}; see~\cite{euclid}.
\end{proof}
\begin{corollary}\label{cor:double}
  For every integer  $n$ , the number  $2n$  is even.
\end{corollary}
\begin{proof}
  Apply Theorem~\ref{thm:parity} to  $n$  and  $n$ .
\end{proof}

```

We obtain the following results, exactly as without the package:

Lemma 1. *The sum of two even integers is even.*

Lemma 2. *The sum of two odd integers is even.*

Theorem 1. *The sum of two integers of the same parity is even.*

Proof. By Lemmas 1 and 2; see [1]. □

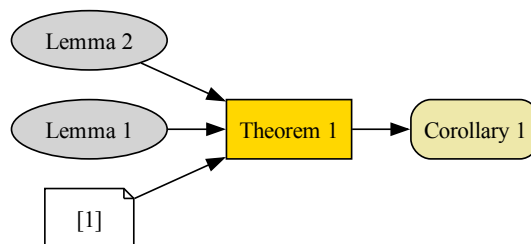
Corollary 1. *For every integer n , the number $2n$ is even.*

Proof. Apply Theorem 1 to n and n . □

In addition, `proofgraph` writes out the dependency graph. With the `autorun` option it is rendered with `Graphviz` and included wherever you call `\proofgraph`:

```
\proofgraph[width=0.7\textwidth]
```

which here produces:



Each cross-reference made inside a proof becomes an arrow *into* the result that proof establishes: the two lemmas point to Theorem 1, which in turn points to Corollary 1. The `\cite` in the proof of Theorem 1 is captured in the same way, thanks to the `cite` option: the external work appears as a distinct node (drawn note-shaped, and labelled here by its printed citation tag, “[1]”, because of `citelabel=tag`) with an arrow into the theorem it helps prove. Moreover, since

this manual loads `hyperref` and `TikZ`, the graph’s nodes are *clickable*: try clicking one to jump to the result it represents (see `\proofgraph` and the `hyperlinks` option).

A handful of commands tune such a graph: `\proofgraphstyle` sets the appearance of each kind of node (the colours here come from it); `\uses` and `\proofgraphedge` record a dependency that no visible `\ref` expresses; `\proofof` attaches a proof to its result; `\proofgraphuntrack` keeps an environment out of the graph; and `\proofgraphignore` and `\proofgraphexclude` drop an unwanted edge or node. The next section puts several of these to work on a real article; Section 3 then documents them, and the package options, in full.

2 A real-world example

The introductory example is deliberately tiny. To show what `proofgraph` produces on a genuine article, the graph in Figure 1 is the one obtained from *Connecting Knowledge Compilation Classes and Width Parameters* [2]. It has 62 numbered results, drawn from five environments (`result`, `theorem`, `proposition`, `corollary` and `lemma`), depends on 19 external works cited inside proofs, and contains 106 edges in all.

The graph was produced from the paper essentially as published. Beyond loading the package, the only additions were the following.

- The `cite` option was switched on, and citation nodes labelled by their printed tag, by loading the package as `\usepackage[cite=true,citelabel=tag]{proofgraph}`. The `\cites` made inside proofs then appear as the note-shaped nodes (“[17]”, “Theorem 4.10, [46]”...) along the left of the graph; a work cited for two different results, or for two of its own theorems, gives one node per `\cite` note, so the 19 works appear as 32 nodes.
- One environment was kept out of the graph with a single `\proofgraphuntrack{observation}` (a lone preliminary observation that nothing else uses).
- The five result kinds were styled, with decreasing prominence, by five `\proofgraphstyle` declarations:

```
\proofgraphstyle{result}{shape=box,style="filled,bold",%
                        fillcolor=orange,peripheries=2}
\proofgraphstyle{theorem}{shape=box,style=filled,%
                        fillcolor=gold,penwidth=2}
\proofgraphstyle{corollary}{shape=box,style="rounded,filled",%
                        fillcolor=palegoldenrod}
\proofgraphstyle{proposition}{shape=box,style="rounded,filled",%
                        fillcolor=lightblue}
\proofgraphstyle{lemma}{shape=ellipse,style=filled,%
                        fillcolor=lightgray}
```

- Eleven `\proofgraphedge` commands, recording 27 edges in total, were added for the dependencies that no `\ref` inside a `proof` expresses: eight “obvious” corollaries stated without a `proof` environment

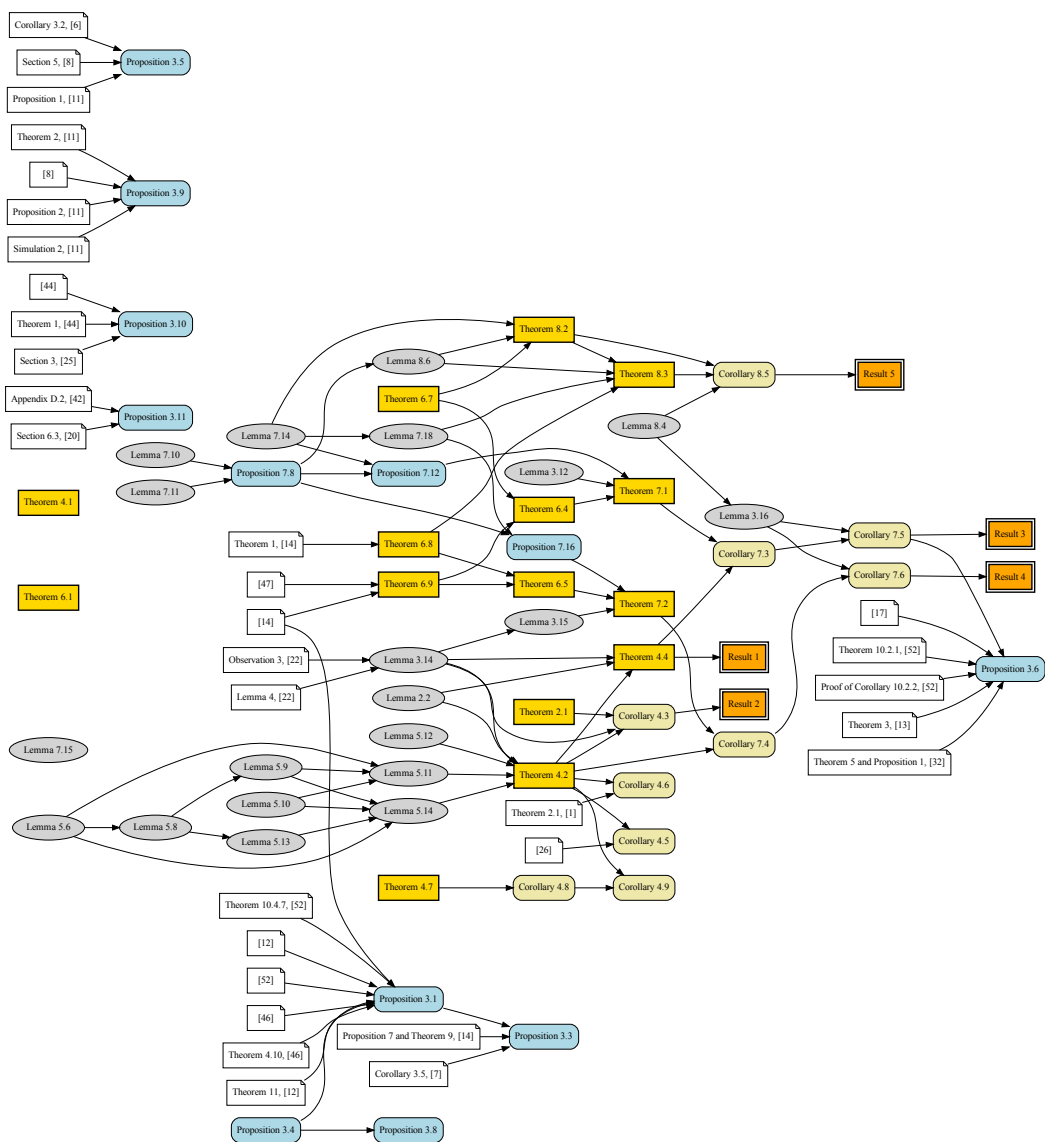


Figure 1: The graph `proofgraph` produces for [2]. The colours and shapes make the structure legible, with the orange double-bordered introduction **results** on the right, fed by the gold theorems, the blue and pale-gold propositions and corollaries, and the grey lemmas.

(`\proofgraphedge{cor:upper_bound}{thm:upper_bound,...}`), and three theorems whose proof is developed across a whole section rather than inside a single `proof`.

Nothing else was annotated: every other edge, and the numbering of all 62 nodes, was inferred automatically from the ordinary cross-references the proofs already contained. In particular, no `\uses`, `\proofof`, `\proofgraphignore` or `\proofgraphexclude` was needed. The five `result` environments that restate the headline results in the introduction (`\begin{result}[(Corollary~\ref{cor:obdd_omega})]...`) are linked to the body results they preview through the `\ref` in their optional title, captured automatically (see the note on reference-carrying titles in Section 3). Two compilation runs (plus `BITEX`, needed once for the `tag` labels) suffice; the `.dot` file is then rendered with `Graphviz`, `dot -Tpdf main.dot -o main.pdf`, as with any other document.

3 Usage

3.1 Default behaviour

Load `proofgraph` in the preamble. It only needs to come *after* whatever provides your theorem environments, that is, after `\usepackage{amsthm}` if you load it yourself (some classes, such as `lipics` or `acmart`, provide theorems for you, in which case there is nothing to load first), and *before* the `\newtheorem` commands that declare your theorem-like environments, so that `proofgraph` sees them as they are created:

```
\usepackage{amsthm}      % or nothing, if your class provides theorems
\usepackage{proofgraph}
\newtheorem{theorem}{Theorem}
\newtheorem{lemma}{Lemma}
```

Then write your document as usual. After two compilation runs, a file `\jobname.dot` is produced. Render it with `Graphviz`:

```
dot -Tpdf myfile.dot -o mygraph.pdf
```

Every *numbered* result of a theorem-like environment you declare becomes a node, and every cross-reference (`\ref`, `\cref...`) inside a proof becomes an edge into the result that proof establishes, with no annotation. The commands captured are `\ref`, `\cref` and `\Cref` (`cleveref`), `\autoref` (`hyperref`), `\eqref` (`amsmath`), `\vref` (`varioref`) and `\zceref` (`zref-clever`, in every form: starred, with options, and with a comma-separated list of labels), each of them whenever it is defined. Purely page-related references (`\pageref`, `\cpageref`, `\zcpageref`) are not captured, as they point at a place rather than at a result; `\uses` records a dependency they would have expressed. Nodes are labelled by the result's formatted name and number ("Theorem 1"), not the environment name. Unnumbered (starred) results are skipped, as they are usually expository or repeated statements; `\proofgraphtrack` draws a chosen one anyway. A cross-reference in the *optional title* of a result is captured the same way: `\begin{theorem}[(generalises~\ref{thm:x})]` in a result labelled $\langle A \rangle$ adds

an edge from $\langle A \rangle$ to `thm:x`, handy for restatements (e.g., an introduction that previews results with `\begin{result}[(Corollary~\ref{cor:y})]`).

3.2 Recording dependencies by hand

`\uses{<label>}` Inside a proof, records a dependency on the result labelled $\langle label \rangle$ even when no visible reference is made. A comma-separated list is accepted, as in `\uses{lem:a,lem:b}`.

`\proofgraphedge{<A>}{<labels>}` Adds edges recording that result $\langle A \rangle$ depends on the result(s) $\langle labels \rangle$ (a comma-separated list), like a `\uses` but stated outside any proof. Handy for an “obvious” corollary that relies on earlier results without a `proof` environment, as in `\proofgraphedge{cor:x}{thm:y,lem:z}`. It may appear anywhere; all labels are resolved when the graph is written.

`\proofof{<label>}` By default a proof is attached to the most recently started result. A detached proof whose optional title names its result is attached automatically: `\begin{proof}` with `[of Theorem~\ref{thm:x}]` attaches the proof to `thm:x` (for the `amsthm` proof environment, for `proof` defined as a theorem in the Springer classes, and for deferred proofs as with `apxproof`). When none of this applies, `\proofof` inside the proof pins it to the result labelled $\langle label \rangle$.

3.3 Choosing what appears in the graph

Result environments are detected automatically; these commands adjust which results, and which edges, are kept.

`\proofgraphtrack{<names>}` Forces the listed environment types to be tracked, overriding the default exclusions, which are `definition`, `example`, `remark` and `note`. Use it in the preamble.

`\proofgraphuntrack{<names>}` Excludes further environment types, as in `\proofgraphuntrack{observation}`. Use it in the preamble.

`\proofgraphignore{<from>}{<to>}` Suppresses the edge from $\langle from \rangle$ to $\langle to \rangle$ (e.g., an unwanted forward citation).

`\proofgraphexclude{<label>}` Removes the result labelled $\langle label \rangle$, together with all its incident edges, from the graph.

`\proofgraphcitecommand{<names>}` Captures further citation commands, given as a comma-separated list of names without their backslash, as in `\proofgraphcitecommand{citeauthor,citeyear}`. Use it in the preamble. Any command taking the usual $\langle cs \rangle * [\langle pre \rangle] [\langle post \rangle] \{ \langle keys \rangle \}$ arguments works, including one you define yourself. Naming a command the document does not define is harmless.

It cannot help with the `biblatex multicite` commands (`\cites`, `\parencites`, `\textcites`), whose $(\langle pre \rangle)(\langle post \rangle)$ syntax and repeated key groups `proof-graph` does not parse; cite such works individually, or record them with `\uses`.

3.4 Styling the graph

The attributes are passed verbatim to Graphviz; see its attribute reference, <https://graphviz.org/doc/info/attrs.html>, for the node attributes available (and <https://graphviz.org/doc/info/shapes.html> for the shapes).

`\proofgraphstyle{<env>}{<attrs>}` Sets Graphviz node attributes for all results of the environment `<env>`, e.g., `shape=ellipse` or `style=filled,fillcolor=gold`.

`\proofgraphstylecite{<attrs>}` Sets the Graphviz attributes of the external citation nodes added by the `cite` option (default `shape=note`), as in `\proofgraphstylecite{shape=box,style=dashed,color=gray}`.

3.5 Embedding the rendered graph

`\proofgraph[<options>]` Includes the rendered graph image at this point, passing the optional `<options>` to the underlying `\includegraphics` (e.g., `\proofgraph[width=\linewidth]`). This is the only feature that needs `graphicx`, which the package therefore does not load on its own: a document that uses `\proofgraph` must load `graphicx` itself (loading `tikz` for the `hyperlinks` overlay also suffices). With the `autorun` option, Graphviz is run at the end of a compilation in which the graph changed (shell-escape required) to produce that image, so `\proofgraph` embeds the graph from the previous run and is up to date after two runs. Without `autorun`, no Graphviz is run: `\proofgraph` embeds a pre-existing `\jobname-proofgraph.<format>` file if you have rendered one under that name, and otherwise warns. When `hyperref` and `TikZ` are both loaded (with `hyperlinks` on and the `-Tplain` file from `autorun` present), every node becomes an internal hyperlink to the result it represents: Graphviz writes the node positions to that `-Tplain` file and `\proofgraph` overlays a transparent `\hyperref` area on each node (needed because the link annotations Graphviz puts in the `.pdf` are discarded by `\includegraphics`).

3.6 Options

selfloops Either `remove` (default), dropping edges from a result to itself, or `keep`, retaining them.

autorun Boolean (default false). Runs Graphviz automatically at the end of the run; requires compilation with shell-escape enabled. Graphviz is run only when the graph, the engine or the format has changed since the last rendering, so that repeated compilations converge and a build tool such as `latexmk` settles instead of rerunning indefinitely.

engine Graphviz layout engine (default `dot`); any Graphviz engine works, such as `neato`, `fdp`, `sfdp`, `circo` or `twopi` (see <https://graphviz.org/docs/layouts/>).

format Output format for `autorun` and `\proofgraph` (default `pdf`); any Graphviz output format works, such as `png`, `svg` or `ps` (see <https://graphviz.org/docs/outputs/>).

hyperlinks Boolean (default true). Makes the nodes of an embedded graph clickable when `hyperref` and `TikZ` are available (see `\proofgraph`); set it false to embed the graph as a plain image.

direction Either `usedby` (default), orienting an edge from the result that is used to the result that uses it (so an arrow `a->b` reads “a is used by b”), or `uses`, the reverse.

rankdir Graphviz graph direction (default LR).

cite Boolean (default false). Also captures `\cite` inside proofs as dependencies on external work, drawn as distinct nodes (Section 2). A `\cite` note is kept: `\cite[Thm~3]{key}` labels the node “Thm 3, ...”. In the two-optional-argument form of `natbib` and `biblatex`, `\cite[see][Thm~3]{key}`, the post-note likewise labels the node and the pre-note is ignored. Every citation command of `LATEX`, `natbib` and `biblatex` that cites a work is captured the same way, whenever the document defines it, starred variants included: `\citet`, `\citep`, `\citeat`, `\citealp`, `\parencite`, `\textcite`, `\autocite`, `\footcite`, `\footcitettext`, `\smartcite`, `\supercite`, `\fullcite` and `\footfullcite`, together with the commands printing only a fragment of a citation (`\citeauthor`, `\citefullauthor`, `\citettitle`, `\citeyear`, `\citeyearpar`, `\citedate`, `\citeurl`, `\citenum`) and all their capitalised forms. A work invoked twice in one proof, by `\citeauthor` and by `\cite` say, still yields a single edge. `\proofgraphcitecommand` adds any further command.

citelabel Either `key` (default), labelling citation nodes by their `BIBTEX` key, or `tag`, using the printed citation tag instead (“[1]”, “[Knu74]”, “[Abiteboul et al. 1995]”...), which is only known after a first run (so it needs two runs). It supports the standard, `hyperref`-wrapped and `natbib` (numeric and author–year) bibliographies. `biblatex` is supported in part: the tag is recovered for its numeric styles ([1]) and its alphabetic ones ([Knu68]), but not for its author–year styles, whose tag is a formatted name list that `biblatex` exposes as no single field. Whenever no clean tag can be recovered the node keeps the key, which is always a correct, if terser, label.

file Base name of the output file (default `\jobname`).

3.7 Use with `apxproof`

`apxproof` moves proofs to the appendix and typesets them there, so the references a proof makes are only executed at that later point. To attribute them to the right result, `proofgraph` relies on the hook `\apxproofhook`, which `apxproof` fires inside each deferred proof. This hook is available in `apxproof v1.4.1` and later; load the two packages in either order and no further setup is needed.

With an *earlier* `apxproof` the document still compiles cleanly and every result still becomes a node, but *no proof dependencies are captured for the proofs deferred to the appendix*, because `proofgraph` cannot tell which appendix proof belongs to which result (dependencies from any proofs left in the main text are still captured as usual). In that case, add the deferred ones manually with `\uses` (and `\proofof` to pin a proof to its result), or upgrade `apxproof`.

4 Limitations

- Environments declared through `\newtheorem` (or `\spnewtheorem`) *after* the package is loaded are tracked automatically. Environments predefined by the document class *before* that (as in `lipics`, `acmart` or the Springer classes) are also detected, provided their name is one of the common result names `proofgraph` probes at `\begin{document}` (`theorem`, `lemma`, `proposition`, `corollary`...); an environment with an unusual name predefined before the package is loaded must be added with `\proofgraphtrack`.
- Results must be labelled for references to them to be resolved.
- Two compilation runs are required, as for any cross-reference.

References

- [1] Euclid. *Elements*, Book IX. c. 300 BCE.
- [2] A. Amarilli, F. Capelli, M. Monet, and P. Senellart. Connecting knowledge compilation classes and width parameters. *Theory of Computing Systems*, **64**(5):861–914, 2020. doi:[10.1007/s00224-019-09930-2](https://doi.org/10.1007/s00224-019-09930-2).

5 Implementation

The format required is that of October 2020, the first to provide `\NewDocumentCommand` in the kernel; `expl3`, which the package also uses, has been preloaded since the February 2020 release.

```

1 (*package)
2 \NeedsTeXFormat{LaTeX2e}[2020/10/01]
3 \ProvidesPackage{proofgraph}
4 [2026/07/27 v1.0.1-dev Dependency graph of results]

```

5.1 Dependencies

`amsthm` (loaded for its heading hooks) defines the `proof` environment with `\newenvironment`, which aborts with “Command `\proof` already defined” under a class that already provides one (e.g. the Springer `svjour3` `\proof`). When we are about to load `amsthm` ourselves (it is not already loaded), we save the class’s `proof/\endproof` and free them so `amsthm` loads cleanly, then restore the class’s versions afterwards, so its proof styling is kept (and `proofgraph` still hooks that `proof` environment). If `amsthm` is already loaded, `\RequirePackage` is a no-op and `\proof` is left untouched.

```

5 \ifpackageloaded{amsthm}{}{%
6   \ifundefined{proof}{}{%
7     \let\pgph@class@proof\proof \let\pgph@class@endproof\endproof}%
8   \let\proof\relax \let\endproof\relax}
9 \RequirePackage{amsthm}
10 \ifundefined{pgph@class@proof}{}{%
11   \let\proof\pgph@class@proof \let\endproof\pgph@class@endproof}
12 \RequirePackage{etoolbox}
13 \RequirePackage{xstring}
14 \RequirePackage{kvoptions}
15 \RequirePackage{pdftexcmds}

```

`pdftexcmds` provides `\pdf@filemdfivesum`, with which `autorun` recognises an unchanged graph and skips the `Graphviz` run (see `\pgph@maybrender`). On an engine that offers no file checksum the package still works, only without that optimisation.

`graphicx` is needed only to embed a rendered graph with `\proofgraph`; the core `.dot`-writing machinery does not use it. Rather than burden every document with it, we leave it unloaded and check for `\includegraphics` at the point of use (see `\proofgraph`). When the `hyperlinks` overlay is active `tikz` is loaded, which pulls `graphicx` in anyway.

`\pgph@trimsp` An expandable space-trimmer (labels inside a `\cite/\cref/\uses` list arrive with the space after each comma), built on the `expl3` primitive so it is robust to leading spaces and braces.

```

16 \ExplSyntaxOn
17 \cs_new:Npn \pgph@trimsp #1 { \tl_trim_spaces:n {#1} }
18 \ExplSyntaxOff

```

5.2 Options

```

19 \SetupKeyvalOptions{family=pgph,prefix=pgph@}
20 \DeclareStringOption[remove]{selfloops}

```

```

21 \DeclareBoolOption{autorun}
22 \DeclareStringOption[dot]{engine}
23 \DeclareStringOption[pdf]{format}
24 \DeclareStringOption[LR]{rankdir}
25 \DeclareStringOption[usedby]{direction}
26 \DeclareBoolOption{cite}
27 \DeclareStringOption[key]{citelabel}
28 \DeclareBoolOption[true]{hyperlinks}
29 \DeclareStringOption{file}
30 \ProcessKeyvalOptions*
31 \ifx\pgph@file\empty\def\pgph@file{\jobname}\fi

```

5.3 Literal braces

We need catcode-12 braces to write the Graphviz digraph { } block.

```

32 \begingroup
33   \catcode'\[=1 \catcode'\]=2
34   \catcode'\{=12 \catcode'\}=12
35   \gdef\pgph@ob[{}]%
36   \gdef\pgph@cb[{}]%
37 \endgroup

```

5.4 Data structures

A global counter gives each result instance a unique numeric id. Nodes and edges are accumulated in list macros and emitted at end of document, so that editing commands (ignore, exclude, self-loops) can be applied to the complete graph regardless of where they appear.

```

38 \newcount\pgph@idcnt
39 \newcount\pgph@citeslotcnt
40 \def\pgph@nodes{}
41 \def\pgph@edges{}
42 \def\pgph@ignores{}
43 \def\pgph@excludes{}
44 \let\pgph@curresult\empty
45 \let\pgph@curproof\empty

```

`\pgph@auxmap` The label-to-id map. `\label` inside a result records, both in memory and in the `.aux` file, that the user label maps to the current node id, so references resolve independently of `hyperref`.

```

46 \newcommand\pgph@auxmap[2]{\global\@namedef{pgph@map@#1}{#2}%
47   \ifcsname pgph@rmap@#2\endcsname\else
48     \global\@namedef{pgph@rmap@#2}{#1}\fi}
49 \newcommand\pgph@setmap[2]{%
50   \edef\pgph@tmpid{#2}%
51   \global\expandafter\edef\csname pgph@map@#1\endcsname{\pgph@tmpid}%
52   \ifcsname pgph@rmap@\pgph@tmpid\endcsname\else
53     \global\expandafter\def\csname pgph@rmap@\pgph@tmpid\endcsname{#1}%
54   \fi
55   \protected@write\@auxout{}\string\pgph@auxmap{#1}{\pgph@tmpid}}%
56 }

```

5.5 Registering result environments

`\pgph@register` A result environment is tracked by installing a begin-hook that records a node. In every declaration command (`\newtheorem`, `\spnewtheorem`) the environment *name* is the first mandatory argument, so registration does not need to parse the remaining (varied) arguments. Each name met is queued as a *candidate* (deduplicated); the begin-hooks themselves are installed only at `\begin{document}`, once the inclusion and exclusion lists are settled, so the customisation commands below may appear anywhere in the preamble.

```

57 \let\pgph@candidates\@empty
58 \newcommand\pgph@register[1]{%
59   \edef\pgph@thisname{#1}%
60   \IfBeginWith\pgph@thisname{axp@}{-}{%
61     \ifcsname pgph@cand@\pgph@thisname\endcsname\else
62       \global\@namedef{pgph@cand@\pgph@thisname}{-}%
63       \ifx\pgph@candidates\@empty
64         \global\let\pgph@candidates\pgph@thisname
65       \else
66         \xdef\pgph@candidates{\pgph@candidates,\pgph@thisname}%
67       \fi
68     \fi
69   }%
70 }
```

We wrap `\newtheorem` (and, for Springer classes, `\spnewtheorem`) to register each newly declared environment, replaying the original command so its own argument parsing is untouched. The starred (unnumbered) forms are *not* auto-registered: a results graph wants numbered statements, and these forms are commonly used for unnumbered repeats of a result (e.g., the appendix copies produced by `apxproof` and `myproofs`), which would otherwise appear as spurious unnumbered duplicate nodes. Use `\proofgraphtrack` to track an unnumbered environment deliberately.

```

71 \let\pgph@orig@newtheorem\newtheorem
72 \renewcommand\newtheorem{\@ifstar{\pgph@nt@star}{\pgph@nt@plain}}
73 \newcommand\pgph@nt@plain[1]{%
74   \pgph@register{#1}\pgph@orig@newtheorem{#1}}
75 \newcommand\pgph@nt@star[1]{\pgph@orig@newtheorem*{#1}}
76 \ifdefined\spnewtheorem
77   \let\pgph@orig@spnewtheorem\spnewtheorem
78   \renewcommand\spnewtheorem{%
79     \@ifstar{\pgph@spnt@star}{\pgph@spnt@plain}}
80   \newcommand\pgph@spnt@plain[1]{%
81     \pgph@register{#1}\pgph@orig@spnewtheorem{#1}}
82   \newcommand\pgph@spnt@star[1]{\pgph@orig@spnewtheorem*{#1}}
83 \fi
```

`\proofgraphtrack`
`\proofgraphuntrack`
`\proofgraphtrack`
`\proofgraphuntrack`

Result environments are detected automatically, but the analysis can be tuned. `\proofgraphtrack{⟨names⟩}` forces the listed environments to be tracked, overriding any exclusion; `\proofgraphuntrack{⟨names⟩}` excludes the listed environment *types*. Expository environments are excluded by default: `definition`, `example`, `remark`, `note`. Both commands belong in the

preamble. `\pgph@defaultenvs` is the list of class-predefined names probed at `\begin{document}`; `\pgph@defaulttexcl` seeds the exclusion set.

```

84 \newcommand\pgph@defaultenvs{theorem,lemma,proposition,corollary,%
85   definition,conjecture,claim,fact,remark,example,observation,%
86   notation,problem,question,property,assumption,hypothesis,principle}
87 \newcommand\pgph@defaulttexcl{definition,example,remark,note}
88 \newcommand\proofgraphtrack[1]{%
89   \@for\pgph@one:=#1\do{%
90     \csundef{pgph@xtype@\pgph@one}%
91     \global\@namedef{pgph@force@\pgph@one}{}%
92     \expandafter\pgph@register\expandafter{\pgph@one}%
93   }%
94 }
95 \newcommand\proofgraphuntrack[1]{%
96   \@for\pgph@one:=#1\do{%
97     \csundef{pgph@force@\pgph@one}%
98     \global\@namedef{pgph@xtype@\pgph@one}{}%
99   }%
100 }
101 \expandafter\proofgraphuntrack\expandafter{\pgph@defaulttexcl}

```

Environments predefined by the document class (acmart, lipics, Springer classes) are declared before the package is loaded, so the wrappers above never see them. At `\begin{document}` we add every common result name that exists to the candidate list, then install a begin-hook for each candidate that is forced or not excluded.

```

102 \newcommand\pgph@dohook[1]{%
103   \ifcsname pgph@hooked@#1\endcsname\else
104     \global\@namedef{pgph@hooked@#1}{}%
105     \AtBeginEnvironment{#1}{\pgph@beginresult{#1}}%
106   \fi
107 }
108 \newcommand\pgph@maybehook[1]{%
109   \ifcsname pgph@force@#1\endcsname
110     \pgph@dohook{#1}%
111   \else
112     \ifcsname pgph@xtype@#1\endcsname\else\pgph@dohook{#1}\fi
113   \fi
114 }
115 \newcommand\pgph@resolveregistration[1]{%
116   \@for\pgph@one:=#1\do{%
117     \ifcsname pgph@one\endcsname%
118       \expandafter\pgph@register\expandafter{\pgph@one}\fi}%
119   \ifx\pgph@candidates\@empty\else
120     \@for\pgph@one:=\pgph@candidates\do{%
121       \expandafter\pgph@maybehook\expandafter{\pgph@one}}%
122   \fi
123 }
124 \AtBeginDocument{%
125   \expandafter\pgph@resolveregistration\expandafter{\pgph@defaultenvs}

```

`\pgph@beginresult` Start a result: allocate an id and record the node. The node is keyed and styled by `\pgph@hooknames` the environment name (third argument, also a fallback label), but displayed by the `\pgph@recordresult` result's *formatted* name (“Theorem”, “Lemma”), captured from the heading. We

get that name by locally wrapping the heading macros, whose first two arguments are the formatted name and the number. Which macro carries the *optional note* (where a restatement's cross-reference lives) depends on the setup: `amsthm`, which `proofgraph` always loads, routes every heading through `\@begintheorem`, defined as `#1#2[#3]` with the note as the bracketed `#3` (and leaves `\@opargbegintheorem` `\relax`); the L^AT_EX kernel instead uses a two-argument `\@begintheorem` with a separate three-argument `\@opargbegintheorem` for the note; the Springer classes use `\@spbegintheorem`/`\@spopargbegintheorem`. We detect the `amsthm` form (`\@opargbegintheorem` is `\relax`) and read the bracketed note there, otherwise we hook `\@opargbegintheorem`; either way the note goes to `\pgph@scanrefs`. These redefinitions are confined to the current environment group, so they revert at `\end`. We also locally redefine `\label` to populate the map.

Nodes are accumulated in *reverse* order of appearance (`\pgph@prependnode`). With `rankdir=LR`, Graphviz stacks disconnected components vertically in the order they are read, from the bottom up; emitting the results last-to-first therefore lays them out top-to-bottom in document order (first result on top), while connected results stay grouped. This is a layout tendency, not a guarantee, but needs no `pack` trickery.

```

126 \newcommand\pgph@prependnode[1]{%
127   \xdef\pgph@nodes{\unexpanded{#1}\unexpanded\expandafter{\pgph@nodes}}%
128 }
129 \newcommand\pgph@beginresult[1]{%
130   \global\advance\pgph@idcnt\@ne
131   \xdef\pgph@curresult{\the\pgph@idcnt}%
132   \protected@edef\pgph@tmp{%
133     \noexpand\pgph@node{\pgph@curresult}{#1}{#1}}%
134   \expandafter\pgph@prependnode\expandafter{\pgph@tmp}%
135   \pgph@hooknames
136   \ifx\label\pgph@maplabel\else\let\pgph@savedlabel\label\fi
137   \let\label\pgph@maplabel
138 }
139 \newcommand\pgph@hooknames{%
140   \ifdefined\@begintheorem
141     \let\pgph@orig@bt\@begintheorem
142     \ifx\@opargbegintheorem\relax
143       \def\@begintheorem##1##2[#3]{%
144         \pgph@recordresult{##1}{##2}\pgph@scanrefs{##3}%
145         \pgph@orig@bt{##1}{##2}[##3]}%
146     \else
147       \def\@begintheorem##1##2{%
148         \pgph@recordresult{##1}{##2}\pgph@orig@bt{##1}{##2}}%
149     \fi
150   \fi
151   \ifdefined\@opargbegintheorem\ifx\@opargbegintheorem\relax\else
152     \let\pgph@orig@obt\@opargbegintheorem
153     \def\@opargbegintheorem##1##2##3{%
154       \pgph@recordresult{##1}{##2}\pgph@scanrefs{##3}%
155       \pgph@orig@obt{##1}{##2}[##3]}%
156   \fi\fi
157   \ifdefined\@spbegintheorem
158     \let\pgph@orig@spbt\@spbegintheorem
159     \def\@spbegintheorem##1##2##3##4{%

```

```

160      \pgph@recordresult{##1}{##2}\pgph@orig@spbt{##1}{##2}{##3}{##4}}%
161    \fi
162    \ifdefined\@spopargbegintheorem
163      \let\pgph@orig@spobt\@spopargbegintheorem
164      \def\@spopargbegintheorem##1##2##3##4##5{%
165        \pgph@recordresult{##1}{##2}\pgph@scanrefs{##3}%
166        \pgph@orig@spobt{##1}{##2}{##3}{##4}{##5}}%
167    \fi
168 }

```

`\pgph@scanrefs` A result whose optional title carries a cross-reference, e.g., a `result` whose title is `[(Corollary~\ref{cor:x})]`, is understood to restate or build on that result: we record an edge from the current result to each label referenced in the title. We capture these by typesetting the title once into a discarded box with the reference-capturing commands installed and `\pgph@curproof` pointing at the current result; `\pgph@curproof` is saved and restored, so this affects nothing outside the title.

```

169 \newcommand\pgph@scanrefs[1]{%
170   \ifx\pgph@curresult\@empty\else
171     \begingroup
172       \global\let\pgph@savedit\pgph@curproof
173       \global\let\pgph@curproof\pgph@curresult
174       \pgph@installrefs
175       \setbox\z@\hbox{#1}%
176       \global\let\pgph@curproof\pgph@savedit
177     \endgroup
178   \fi
179 }

```

The formatted name is the first argument of every heading macro (`\@begintheorem/\@opargbegintheorem` for `amsthm`/`\newtheorem`, `\@spbegintheorem/\@spopargbegintheorem` for Springer's `\spnewtheorem`); the second argument is the number. We use that second argument only to tell *whether* the result is numbered (it is empty for the starred, unnumbered forms): an unnumbered result records no number, so it is dropped at emission unless tracked explicitly. The number *value*, when present, is read from `\@currentlabel` rather than from that argument, which some classes pollute (e.g., `lipics` injects `\advance\par@deathcycles\@one`). `\@currentlabel`, set by the result's `\refstepcounter` just before the heading, is the clean number and needs no `\label`. Values are reduced to plain strings now, with fragile wrappers neutralized (notably `apxproof`'s forward-link). The first heading wins (guarded on the name), ignoring nested headings. The emptiness of the number argument is tested without expanding it, so a polluted (but non-empty) number does not trip the test.

```

180 \newcommand\pgph@recordresult[2]{%
181   \ifcsname pgph@name@\pgph@curresult\endcsname\else
182     \begingroup
183       \let\protect\@empty
184       \def\axp@forward@link##1##2{##2}%
185       \edef\pgph@tmpname{#1}%
186       \global\expandafter\let\csname pgph@name@\pgph@curresult\endcsname
187         \pgph@tmpname
188       \global\@namedef{pgph@seen@\pgph@curresult}{}%

```

```

189 \def\pgph@tmphn{#2}%
190 \ifx\pgph@tmphn\@empty\else
191 \edef\pgph@tmpnum{\@currentlabel}%
192 \global\expandafter\let\csname pgph@num@\pgph@curresult%
193 \endcsname\pgph@tmpnum
194 \fi
195 \endgroup
196 \fi
197 }

```

`\pgph@maplabel` records the label-to-node mapping. As a fallback for classes whose headings pass through none of the hooked heading macros (so no heading was `seen`), it also captures the number from `\@currentlabel`. We do *not* do this once a heading was seen: there, an absent number means the result is genuinely unnumbered (a starred form), and `\@currentlabel` could be a stale value from an earlier result. The value is expanded to a plain string *now*, inside a group where fragile wrappers are neutralized, so that nothing dangerous survives to the emission pass: in particular, under `apxproof` forward-linking, `\@currentlabel` holds `\axp@forward@link{<target>}{<number>}`, whose hyper-link machinery must never be expanded in a non-typesetting context (it would run away). Reducing it to its number argument keeps it harmless.

`\pgph@maplabel`

```

198 \newcommand\pgph@maplabel[1]{%
199 \pgph@setmap{#1}{\pgph@curresult}%
200 \ifcsname pgph@num@\pgph@curresult\endcsname\else
201 \ifcsname pgph@seen@\pgph@curresult\endcsname\else
202 \begingroup
203 \let\protect\@empty
204 \def\axp@forward@link##1##2{##2}%
205 \edef\pgph@tmpnum{\@currentlabel}%
206 \global\expandafter\let\csname pgph@num@\pgph@curresult%
207 \endcsname\pgph@tmpnum
208 \endgroup
209 \fi
210 \fi
211 \pgph@savdlabell{#1}%
212 }

```

5.6 Hooking proofs

Inside a proof we redefine the reference commands to capture edges from the proved result.

```

213 \newcommand\pgph@adddedge[2]{%
214 \protected@edef\pgph@tmpedge{\noexpand\pgph@edge{#1}{#2}}%
215 \expandafter\g@addto@macro\expandafter\pgph@edges%
216 \expandafter{\pgph@tmpedge}%
217 }
218 \newcommand\pgph@recordref[1]{%
219 \ifx\pgph@curproof\@empty\else
220 \@for\pgph@one:=#1\do{%
221 \edef\pgph@k{\expandafter\pgph@trimsp\expandafter{\pgph@one}}%
222 \pgph@adddedge{\pgph@curproof}{\pgph@k}}%

```

```

223 \fi
224 }

```

`\pgph@beginproof` Attach the proof to the current result and install the capturing versions of the reference commands (only those that exist).

The capturing commands are `\protected` so that, when a reference appears in a moving argument inside a proof (e.g., a `\caption`), they do not expand into the `.aux/.lof` and leak internal tokens.

```

225 \protected\def\pgph@cap@ref#1{\pgph@recordref{#1}\pgph@saveref{#1}}
226 \protected\def\pgph@cap@cref#1{\pgph@recordref{#1}\pgph@saveref{#1}}
227 \protected\def\pgph@cap@Cref#1{\pgph@recordref{#1}\pgph@saveref{#1}}
228 \protected\def\pgph@cap@autoref#1{%
229   \pgph@recordref{#1}\pgph@saveref{#1}}
230 \protected\def\pgph@cap@eqref#1{%
231   \pgph@recordref{#1}\pgph@saveref{#1}}
232 \protected\def\pgph@cap@vref#1{\pgph@recordref{#1}\pgph@saveref{#1}}

```

`zref-clever`'s `\zcref` is the one captured command that is not simply `\cs{<labels>}`: it is `\zcref*[{<options>}]{<labels>}`, and both the star and the options have to be handed on unchanged. Its mandatory argument is a comma-separated list, which `\pgph@recordref` already splits.

```

233 \NewDocumentCommand\pgph@cap@zcref{s0{}m}{%
234   \pgph@recordref{#3}%
235   \IfBooleanTF{#1}{\pgph@saveref@zcref*{#2}{#3}}{\pgph@saveref@zcref{#2}{#3}}%
236 }
237 \newcommand\pgph@install[1]{%
238   \ifcsname #1\endcsname
239     \expandafter\ifx\csname #1%
240       \expandafter\endcsname\csname pgph@cap@#1\endcsname
241     \else
242       \expandafter\let\csname pgph@saveref@#1%
243       \expandafter\endcsname\csname #1\endcsname
244       \expandafter\let\csname #1%
245       \expandafter\endcsname\csname pgph@cap@#1\endcsname
246     \fi
247 \fi
248 }

```

The set of captured commands is needed in three places (a proof, the optional title of a result, and an `apxproof` deferred proof), so it is named once here: adding a command to the list must not be a matter of remembering three call sites.

```

249 \newcommand\pgph@installrefs{%
250   \pgph@install{ref}\pgph@install{cref}\pgph@install{Cref}%
251   \pgph@install{autoref}\pgph@install{eqref}\pgph@install{vref}%
252   \pgph@install{zcref}%
253 }
254 \newcommand\pgph@beginproof{%
255   \let\pgph@curproof\pgph@curresult
256   \pgph@installrefs
257   \ifpgph@cite\pgph@installcite\fi
258   \pgph@hookproofhead
259 }

```

When the `proof` environment is itself a theorem-like environment (as with Springer's `\spnewtheorem*{proof}`), its optional title is set by `\@opargbegintheorem/\@spopargbegintheorem`. We wrap these for the duration of the proof so that a title such as `[of Theorem~\ref{thm:x}]` sets `\proofof{thm:x}`, attributing the proof to its result.

```

260 \newcommand\pgph@hookproofhead{%
261   \ifdefined\@opargbegintheorem
262     \let\pgph@orig@obtp\@opargbegintheorem
263     \def\@opargbegintheorem##1##2##3{%
264       \pgph@scanproofarg{##3}\pgph@orig@obtp{##1}{##2}{##3}}%
265   \fi
266   \ifdefined\@spopargbegintheorem
267     \let\pgph@orig@spobtp\@spopargbegintheorem
268     \def\@spopargbegintheorem##1##2##3##4##5{%
269       \pgph@scanproofarg{##3}%
270       \pgph@orig@spobtp{##1}{##2}{##3}{##4}{##5}}%
271   \fi
272   \ifdefined\@Opargbegintheorem
273     \let\pgph@orig@Obtp\@Opargbegintheorem
274     \def\@Opargbegintheorem##1##2##3##4{%
275       \pgph@scanproofarg{##2}\pgph@orig@Obtp{##1}{##2}{##3}{##4}}%
276   \fi
277 }
```

`\pgph@scanproofarg` A proof whose optional title names its result (a detached proof) has its result given by that title, e.g., `thm:x` for an optional argument `[of Theorem~\ref{thm:x}]`, rather than by the most recently opened result. We capture this by wrapping the `proof` environment so that, when an optional title is given, the cross-reference it contains is turned into a `\proofof` (the title is typeset once into a discarded box with `\ref` and friends redefined). Without an optional title the original environment is used unchanged.

```

278 \NewDocumentCommand\pgph@zcproofof{s0{m}}{\proofof{#3}}
279 \newcommand\pgph@scanproofarg[1]{%
280   \begingroup
281     \let\protect\@empty
282     \def\ref##1{\proofof{##1}}\def\cref##1{\proofof{##1}}%
283     \def\Cref##1{\proofof{##1}}\def\autoref##1{\proofof{##1}}%
284     \let\zceref\pgph@zcproofof
285     \setbox\z@\hbox{#1}%
286   \endgroup
287 }
```

To capture the optional title of a detached `amsthm` proof (an optional `[of Theorem~\ref{thm:x}]`) we redefine the outer `\proof` to read the optional argument and delegate to the original inner macro (saved as `\pgph@proofinner`), so no recursion occurs. We must do this *only* for a genuine optional-argument environment: a package that captures the proof body verbatim (`apxproof`, `myproofs`) redefines `proof` with no optional argument, and wrapping it would break that capture. We recognise the optional-argument form by the `\@protected@testopt` in its definition. When `proof` is instead a theorem-like environment (Springer `\spnewtheorem*{proof}`) its title is already handled by `\pgph@hookproofhead`, so here we just install the begin-hook.

```

288 \newcommand\pgph@wrapproof{%
289   \ifdefined\proof
290     \edef\pgph@proofmeaning{\meaning\proof}%
291     \edef\pgph@testoptstr{\detokenize{\@protected@testopt}}%
292     \IfSubStr\pgph@proofmeaning\pgph@testoptstr
293       {\pgph@redefproof}%
294       {\AtBeginEnvironment{proof}{\pgph@beginproof}}%
295   \fi
296 }
297 \newcommand\pgph@redefproof{%
298   \expandafter\let\expandafter\pgph@proofinner%
299   \csname\string\proof\endcsname
300   \def\proof{\pgph@beginproof
301     \@ifnextchar[{\pgph@proofopt}{\pgph@proofinner[\proofname]}}%
302   \def\pgph@proofopt[##1]{\pgph@scanproofarg{##1}\pgph@proofinner[##1]}%
303 }

```

Packages such as `apxproof` capture the body of a `proof` verbatim (to defer it to the appendix) and replay it there. Patching the `proof` environment then corrupts that capture, and the references are only executed at replay time anyway. So we wrap `proof` only when no such package is active; otherwise the deferred-proof capture (below) takes over. This happens at `\begin{document}`, once we know what is loaded.

```

304 \newif\ifpgph@apxproof
305 \AtBeginDocument{%
306   \@ifpackageloaded{apxproof}%
307   {\pgph@apxprooftrue\pgph@apxinit}%
308   {\pgph@wrapproof}%
309 }

```

`\pgph@apxinit` Under `apxproof`, deferred proofs are typeset in the appendix, where their references finally execute. `apxproof` fires `\apxproofhook` inside each such proof, passing the `exp@r<n>` label of the proved theorem, which (because `apxproof` also attaches that label to the theorem in the main text) is already in our map. We resolve it to the source node and install the reference-capturing commands for the duration of the proof.

```

310 \newcommand\pgph@apxinit{%
311   \def\apxproofhook##1{\pgph@apxproofcapture{##1}}
312 \newcommand\pgph@apxproofcapture[1]{%
313   \edef\pgph@curproof{\pgph@resolve{#1}}%
314   \ifx\pgph@curproof\@empty\else
315     \pgph@installrefs
316     \ifpgph@cite\pgph@installcite\fi
317   \fi
318 }

```

`\pgph@hookcite` Optional citation capture, applied to `\cite` and, when they are defined (`natbib`, `biblatex`), to `\citet` and `\citep` as well, all sharing one capture chain parameterized by the command name: external references become `cite:-`prefixed target labels, drawn later as distinct nodes. The node identity is the pair (key, note): a `\cite[<note>]{<key>}` inside a proof is distinguished from a `\cite` of the same key with a *different* note, so that citing two different results of the same work (say two theorems of one book) yields two nodes rather than one mislabelled with whichever

note was seen first. Each pair is assigned a numeric *slot* (`\pgph@citeslot`); the edge target is `\cite:<slot>`, and the slot remembers the key (for the tag and the bibliography hyperlink) and the note (for the label). The note is reduced to a plain string at capture time (fragile wrappers neutralized, `~` normalised to a space), which also serves as the signature so `[Thm 1]` and `[Thm~1]` coincide. We peek for the optional argument with `\@ifnextchar` rather than declaring one, so that a note-less `\cite{<key>}` is passed on *as written*: re-emitting it as `\cite[]{<key>}` would make the typeset citation grow a spurious empty note (“[1,]”). A second optional argument is peeked for in the same way, to support the two-argument form `\cite[<pre-note>][<note>]{<key>}` of `natbib` and `biblatex`; there the *second* argument is the note recorded for the node (matching the one-argument semantics, where the lone argument is the post-note), while the pre-note (“see”, “e.g.”) qualifies the act of citing rather than identifying the cited result, and is only replayed to the saved `\cite`. Without this peek, the mandatory-argument scan would grab the bare token `[` as the key and typeset the rest of the citation as text, giving confusing errors (“Missing \$ inserted”) deep in the proof body. A star (`\citet*`, `\citep*`: full author list) is likewise peeked for and replayed; it affects rendering only, not the recorded (key, note) pair. The replay head (saved command plus optional star) is frozen in `\pgph@replay` by the star-peek before the arguments are scanned. As in `\pgph@install`, installation is skipped (per command) when the command is already the capture macro; otherwise a proof nested inside another proof would save the capture as the saved command, and the first citation in the inner proof would recurse forever.

```

319 \newcommand\pgph@gxdefcs[2]{%
320   \global\expandafter\edef\csname#1\endcsname{#2}}
321 \newcommand\pgph@cap@citegen[1]{%
322   \def\pgph@citecmd{#1}%
323   \@ifstar{\pgph@cap@cite@x*}{\pgph@cap@cite@x}}
324 \def\pgph@cap@cite@x#1{%
325   \edef\pgph@replay{%
326     \expandafter\noexpand\csname pgph@saved@\pgph@citecmd\endcsname#1}%
327   \@ifnextchar[\pgph@cap@cite@opt\pgph@cap@cite@noopt}
328 \def\pgph@cap@cite@opt[#1]{%
329   \@ifnextchar[\pgph@cap@cite@optii{#1}]{\pgph@cap@cite@opti{#1}}}
330 \def\pgph@cap@cite@opti#1#2{%
331   \pgph@cap@citerec{#1}{#2}\pgph@replay[#1]{#2}}
332 \def\pgph@cap@cite@optii#1[#2]#3{%
333   \pgph@cap@citerec{#2}{#3}\pgph@replay[#1][#2]{#3}}
334 \def\pgph@cap@cite@noopt#1{\pgph@cap@citerec{#1}\pgph@replay{#1}}
335 \newcommand\pgph@cap@citerec[2]{%
336   \ifx\pgph@curproof\empty\else
337     \@for\pgph@one:=#2\do{%
338       \edef\pgph@k{\expandafter\pgph@trimsp\expandafter{\pgph@one}}%
339       \pgph@citeslot{\pgph@k}{#1}%
340       \pgph@addedge{\pgph@curproof}{\cite:\pgph@curslot}}%
341     \fi
342 }
343 \newcommand\pgph@citeslot[2]{%
344   \begingroup
345     \pgph@citesanitize
346     \xdef\pgph@gnotestr{#2}%
347   \endgroup

```

```

348 \edef\pgph@sig{#1@@\detokenize\expandafter{\pgph@gnotestr}}%
349 \ifcsname pgph@cslot@\pgph@sig\endcsname
350 \edef\pgph@curslot{\csname pgph@cslot@\pgph@sig\endcsname}%
351 \else
352 \global\advance\pgph@citeslotcnt\@ne
353 \edef\pgph@curslot{\the\pgph@citeslotcnt}%
354 \pgph@gxdefcs{pgph@cslot@\pgph@sig}{\pgph@curslot}%
355 \pgph@gxdefcs{pgph@cskey@\pgph@curslot}{#1}%
356 \ifx\pgph@gnotestr\@empty\else
357 \pgph@gxdefcs{pgph@citenote@\pgph@curslot}{\pgph@gnotestr}%
358 \fi
359 \fi
360 }

```

`\pgph@citecmds` Every citation command with the signature `\cs*[[pre]][post]{keys}` is captured by the same `\pgph@cap@citegen`, so which commands are captured `\proofgraphcitecommand` is a list rather than code. The default list holds the citation commands of L^AT_EX, natbib and biblatex that cite a work; `\proofgraphcitecommand` adds more. Naming a command that the document never defines costs nothing, as `\pgph@installciteone` installs only over an existing command, so the list may name every spelling a citation package might provide.

The commands printing only a *fragment* of a citation (`\citeauthor`, `\citeyear`...) are in the list as well. They commonly accompany a real citation of the same work, but a repetition is harmless: a (key, note) pair already seen reuses its slot, and `\pgph@writeedge` draws an edge once, so the second mention of a work adds nothing to the graph. Capturing them means that a proof invoking a work by author or by year alone, which does happen, is not silently left without its edge.

The biblatex *multicite* commands (`\cites`, `\parencites`, `\textcites`) cannot be captured at all: their (*pre*)(*post*) syntax and repeated key groups are a different grammar, which `\pgph@cap@citegen` does not parse.

```

361 \newcommand\pgph@citecmds{%
362   cite,Cite,citet,Citet,citep,Citep,%
363   citealt,Citealt,citealp,Citealp,%
364   parencite,Parencite,textcite,Textcite,autocite,Autocite,%
365   footcite,Footcite,footcitetext,smartcite,Smartcite,supercite,%
366   fullcite,footfullcite,%
367   citeauthor,Citeauthor,citefullauthor,citetitle,Citetitle,%
368   citeyear,Citeyear,citeyearpar,citedate,Citedate,citeurl,citenum}
369 \newcommand\proofgraphcitecommand[1]{\g@addto@macro\pgph@citecmds{,#1}}
370 \newcommand\pgph@installcite{%
371   \@for\pgph@conename:=\pgph@citecmds\do{%
372     \edef\pgph@cone{\expandafter\pgph@trimsp\expandafter{\pgph@conename}}%
373     \expandafter\pgph@installciteone\expandafter{\pgph@cone}}%
374 }

```

The capture macro of a command is built on first use rather than declared: it is the same `\pgph@cap@citegen` call for all of them, and building it only for commands the document actually defines keeps the unused names of the list from costing anything. It must be `\gdef`, the installation happening inside the group of a proof.

```

375 \newcommand\pgph@installciteone[1]{%
376   \ifcsname #1\endcsname

```

```

377 \ifcsname pgph@cap@#1\endcsname\else
378 \expandafter\gdef\csname pgph@cap@#1\endcsname{\pgph@cap@citegen{#1}}%
379 \fi
380 \expandafter\ifx\csname #1\expandafter\endcsname
381 \csname pgph@cap@#1\endcsname
382 \else
383 \expandafter\let\csname pgph@saved@#1\expandafter\endcsname
384 \csname #1\endcsname
385 \expandafter\let\csname #1\expandafter\endcsname
386 \csname pgph@cap@#1\endcsname
387 \fi
388 \fi
389 }

```

The biblatex tag recorder is needed only when tags are actually wanted, and is registered at the end of the preamble, once biblatex has certainly been loaded (it is a preamble-only package) but while \AtEveryCitekey still accepts additions.

```

390 \AtEndPreamble{%
391 \ifpgph@cite
392 \ifdefstring{\pgph@citelabel}{tag}%
393 {\@ifpackageloaded{biblatex}{\AtEveryCitekey{\pgph@blxrecord}}{}}}%
394 \fi
395 }

```

5.7 User commands

```

396 \newcommand\uses[1]{\pgph@recordref{#1}}
397 \newcommand\proofof[1]{%
398 \ifcsname pgph@map@#1\endcsname
399 \xdef\pgph@curproof{\csname pgph@map@#1\endcsname}%
400 \fi
401 }
402 \newcommand\proofgraphedge[2]{%
403 \g@addto@macro\pgph@edges{\pgph@labeledge{#1}{#2}}%
404 }
405 \newcommand\pgph@labeledge[2]{%
406 \edef\pgph@ef{\pgph@resolve{#1}}%
407 \ifx\pgph@ef\@empty\else
408 \@for\pgph@one:=#2\do{%
409 \edef\pgph@k{\expandafter\pgph@trimsp\expandafter{\pgph@one}}%
410 \pgph@plainedge{\pgph@ef}{\pgph@k}}%
411 \fi
412 }
413 \newcommand\proofgraphignore[2]{%
414 \g@addto@macro\pgph@ignores{\pgph@ignorerule{#1}{#2}}%
415 }
416 \newcommand\proofgraphexclude[1]{%
417 \g@addto@macro\pgph@excludes{\pgph@excluderule{#1}}%
418 }
419 \newcommand\proofgraphstyle[2]{\global\@namedef{pgph@style@#1}{#2}}

```

External citation nodes (option cite) share one style, held in \pgph@citestyle and overridable with \proofgraphstylecite; it defaults to shape=note.

```

420 \newcommand\pgph@citestyle{shape=note}

```

```
421 \newcommand\proofgraphstylecite[1]{\renewcommand\pgph@citestyle{#1}}
```

5.8 Resolution helpers

These run at end of document, after the .aux has populated the map.

```
422 \newcommand\pgph@resolve[1]{%
423   \ifcsname pgph@map@#1\endcsname\csname pgph@map@#1\endcsname\fi
424 }
425 \newcommand\pgph@ignorerule[2]{%
426   \edef\pgph@f{\pgph@resolve{#1}}\edef\pgph@t{\pgph@resolve{#2}}%
427   \ifx\pgph@f\@empty\else\ifx\pgph@t\@empty\else
428     \@namedef{pgph@ig@\pgph@f @\pgph@t}{}%
429   \fi\fi
430 }
431 \newcommand\pgph@excluderule[1]{%
432   \edef\pgph@x{\pgph@resolve{#1}}%
433   \ifx\pgph@x\@empty\else\@namedef{pgph@ex@\pgph@x}{}\fi
434 }
```

5.9 Emission

`\pgph@dotes` Escape the two characters that are special in a Graphviz double-quoted string, so a theorem name, number or citation note containing them does not corrupt the .dot: a backslash is doubled and a double quote is backslash-escaped (in that order). `\pgph@bschar` and `\pgph@dqchar` hold the catcode-12 backslash and double quote to search for; `xstring`'s expansion mode is saved and restored around the two substitutions.

```
435 \begingroup
436   \catcode'\|=0 \catcode'\|=12 \catcode'\|=12
437   \gdef\pgph@bschar{\}%
438   \gdef\pgph@dqchar{"}%
439 \endgroup
440 \newcommand\pgph@dotes[1]{%
441   \saveexpandmode\expandarg
442   \StrSubstitute{#1}{\pgph@bschar}{\pgph@bschar\pgph@bschar}[#1]%
443   \StrSubstitute{#1}{\pgph@dqchar}{\pgph@bschar\pgph@dqchar}[#1]%
444   \restoreexpandmode}
```

`\pgph@node` Emit one node line, honouring exclusions and per-environment styling. External (`\cite{}`) targets are emitted separately as a pass over edges. An *unnumbered* result (no number captured) is omitted: a results graph is about numbered statements, and unnumbered theorem-like environments are typically expository or repeated copies (e.g., a class-predefined `\spnewtheorem*{claim}`). `\proofgraphtrack{<env>}` overrides this, so a deliberately unnumbered environment is still drawn.

```
445 \newcommand\pgph@node[3]{%
446   \ifcsname pgph@ex@#1\endcsname\else
447     \edef\pgph@nm{\ifcsname pgph@num@#1\endcsname%
448       \csname pgph@num@#1\endcsname\fi}%
449     \ifx\pgph@nm\@empty
450       \ifcsname pgph@force@#2\endcsname
451         \pgph@emitnode{#1}{#2}{#3}%
452       \else
```

```

453     \@namedef{pgph@ex@#1}{}%
454     \fi
455     \else
456         \pgph@emitnode{#1}{#2}{#3}%
457     \fi
458 \fi
459 }
460 \newcommand\pgph@emitnode[3]{%
461     \edef\pgph@s{\ifcsname pgph@style@#2\endcsname
462         , \csname pgph@style@#2\endcsname\fi}%
463     \edef\pgph@dn{\ifcsname pgph@name@#1\endcsname
464         \csname pgph@name@#1\endcsname\else#3\fi}%
465     \edef\pgph@lbl{\pgph@dn\space\pgph@nm}%
466     \pgph@dotes\pgph@lbl
467     \immediate\write\pgph@out{%
468         \space\space"#1" [label="\pgph@lbl"\pgph@s];}%
469     \ifcsname pgph@rmap@#1\endcsname
470         \immediate\write\pgph@mapout{%
471             \string\pgph@nodemap{#1}{\csname pgph@rmap@#1\endcsname}}}%
472     \fi
473 }

```

\pgph@edge Emit one edge, resolving the target label to an id and applying the self-loop, ignore and exclude filters. Targets beginning with **cite:** denote external nodes.

```

474 \newcommand\pgph@edge[2]{%
475     \IfBeginWith{#2}{cite:}%
476         {\pgph@citeedge{#1}{#2}}%
477         {\pgph@plainedge{#1}{#2}}%
478 }
479 \newcommand\pgph@plainedge[2]{%
480     \edef\pgph@t{\pgph@resolve{#2}}%
481     \ifx\pgph@t@empty\else
482         \ifcsname pgph@ex@#1\endcsname\else
483             \ifcsname pgph@ex@\pgph@t\endcsname\else
484                 \ifcsname pgph@ig@#1@\pgph@t\endcsname\else
485                     \ifboolexpr{ test {\ifnumequal{#1}{\pgph@t}}
486                         and test {\ifdefstring{\pgph@selfloops}{remove}} }%
487                         {\pgph@emitedge{#1}{n\pgph@t}}%
488                     \fi\fi\fi
489                 \fi
490 }

```

\pgph@emitedge The relation recorded is always “the result whose proof we are in (the first argument) uses the target (the second)”. The **direction** option chooses the arrow orientation: **direction=usedby** (default) draws *target to result*, so an arrow **a->b** reads “a is used by b” (i.e. b relies on a); **direction=uses** draws *result to target* (“a uses b”). The filters above work on the recorded relation, so they are unaffected by the chosen orientation. **\pgph@writeedge** deduplicates.

```

491 \newcommand\pgph@emitedge[2]{%
492     \ifdefstring{\pgph@direction}{uses}%
493         {\pgph@writeedge{#1}{#2}}%
494         {\pgph@writeedge{#2}{#1}}%
495 }

```

```

496 \newcommand\pgph@writeedge[2]{%
497   \ifcsname pgph@drawn@#1@#2\endcsname\else
498     \@namedef{pgph@drawn@#1@#2}{}%
499     \immediate\write\pgph@out{\space\space"#1" -> "#2";}%
500   \fi
501 }

```

`\pgph@citeedge` A `cite:⟨slot⟩` target: recover the slot’s `BIBTEX` key, declare the external node once per slot, then draw the edge. The node *label* is built by `\pgph@setcitelabel`: it is the key by default, or, with `citelabel=tag`, the printed citation tag (the “[42]”/“[Knu74]” text), recovered either from a tag recorded by `\pgph@blxrecord` under `blatex` or from the `\b@⟨key⟩` macro that `\bibcite` writes to the `.aux` (so two runs are needed, and it falls back to the key when no tag is known, e.g., on the first run or with citation packages neither path covers). The slot’s `\cite` note, when present, is prepended as “⟨note⟩, ...”. The node-to-label map records the *key*, so all slots of one work hyperlink to the same bibliography entry.

```

502 \newcommand\pgph@citeedge[2]{%
503   \ifcsname pgph@ex@#1\endcsname\else
504     \StrBehind{#2}{cite:}\pgph@slot}%
505     \edef\pgph@key{\csname pgph@cskey@\pgph@slot\endcsname}%
506     \ifcsname pgph@extnode@\pgph@slot\endcsname\else
507       \@namedef{pgph@extnode@\pgph@slot}{}%
508       \expandafter\pgph@setcitelabel\expandafter{\pgph@slot}%
509       \pgph@dotes\pgph@clabel
510       \immediate\write\pgph@out{%
511         \space\space"c@\pgph@slot"
512         [label="\pgph@clabel",\pgph@citestyle];}%
513       \immediate\write\pgph@mapout{%
514         \string\pgph@nodemapcite{c@\pgph@slot}{\pgph@key}}%
515     \fi
516     \pgph@emitedge{#1}{c@\pgph@slot}%
517   \fi
518 }
519 \edef\pgph@obraces{\expandafter\@gobble\string\{}}
520 \newif\ifpgph@natnum
521 \newcommand\pgph@nil{}

```

`\pgph@citesanitize` neutralizes the wrappers and spacing cruft that citation labels carry, so that an `\edef` of `\b@⟨key⟩` yields plain text: `hyperref`’s `\hyper@link` (keep its printed last argument) and the `boxing/penalty/\ignorespaces` noise that `natbib` bakes into author lists.

```

522 \newcommand\pgph@citesanitize{%
523   \let\protect\@empty
524   \def\hyper@link[##1]##2##3##4{##4}%
525   \let\unskip\@empty \let\ignorespaces\@empty \let\unhbox\@empty
526   \let\penalty\@gobble \let\@M\@empty \let\voidb@x\@empty
527   \let\nobreakspace\space \let~\space
528   \def\hbox##1{##1}\def\mbox##1{##1}%
529 }

```

A `natbib` `\b@⟨key⟩` is `{⟨num⟩}{⟨year⟩}{⟨short⟩}{⟨long⟩}`. In numbers mode the tag is the number; in author–year mode it is the short author list and the year.

```

530 \def\pgph@natpick#1#2#3#4\pgph@nil{%
531   \ifpgph@natnum\def\pgph@x{#1}\else\def\pgph@x{#3 #2}\fi}

```

```

532 \newcommand\pgph@natdo{\expandafter\pgph@natpick\pgph@x{}{}{}\pgph@nil}

\pgph@natnumset Which of the two natbib modes is in force is read off \ifNAT@numbers, by compar-
ing it with \iftrue. That comparison must live in a macro of its own rather than
inline in \pgph@setcitelabel, because the code around it can be skipped by a
false conditional: skipping does not expand anything, it merely counts conditional
tokens, and the \iftrue here – an operand of \ifx, not a conditional – would be
counted as one more \if to be closed. The skip would then eat one \fi too many
and run past the end of \pgph@setcitelabel into its caller, silently swallowing
the very writes that emit the citation node. Inside a macro the token is never seen
by the skipping scanner.

533 \newcommand\pgph@natnumset{%
534   \pgph@natnumfalse
535   \@ifundefined{ifNAT@numbers}{}{%
536     \expandafter\ifx\csname ifNAT@numbers\endcsname\iftrue
537     \pgph@natnumtrue\fi}%
538 }

\pgph@blxrecord biblatex writes no \b@{key}: its tags are assembled while the citation is type-
set, from data the .bbl holds. We therefore record them as they go by, from
\AtEveryCitekey (installed below), where the entry is the current one and its
fields are readable. labelnumber carries the tag of the numeric styles and
labelalpha that of the alphabetic ones; the author–year styles have neither, as
their tag is a formatted name list rather than a field, and are left to the key fall-
back. A field that survives sanitizing with a backslash still in it is not plain text,
and is likewise declined.

539 \newcommand\pgph@blxrecord{%
540   \begingroup
541   \pgph@citesanitize
542   \edef\pgph@blxkey{\thefield{entrykey}}%
543   \edef\pgph@x{\thefield{labelnumber}}%
544   \ifx\pgph@x@empty \edef\pgph@x{\thefield{labelalpha}}\fi
545   \edef\pgph@dx{\detokenize\expandafter{\pgph@x}}%
546   \ifx\pgph@x@empty\else
547     \IfSubStr\pgph@dx@backslashchar{}{%
548       \pgph@gxdefcs{pgph@blxtag@pgph@blxkey}{\pgph@x}}%
549   \fi
550   \endgroup
551 }

552 \newcommand\pgph@setcitelabel[1]{%
553   \edef\pgph@ckey{\csname pgph@cskey@#1\endcsname}%
554   \edef\pgph@cbase{\pgph@ckey}%
555   \ifdefstring{\pgph@citelabel}{tag}{%
556     \ifcsname pgph@blxtag@pgph@ckey\endcsname
557       \edef\pgph@cbase{[\csname pgph@blxtag@pgph@ckey\endcsname]}%
558     \else
559       \ifcsname b@\pgph@ckey\endcsname
560         \global\let\pgph@cbase\relax
561       \begingroup
562         \pgph@citesanitize
563         \edef\pgph@x{\csname b@\pgph@ckey\endcsname}%
564         \edef\pgph@dx{\detokenize\expandafter{\pgph@x}}%

```

```

565     \IfBeginWith\pgph@dx\pgph@obrace{%
566         \pgph@natnumset
567         \pgph@natdo
568         \edef\pgph@dx{\detokenize\expandafter{\pgph@x}}%
569     }{}%
570     \ifx\pgph@x\@empty\else
571         \IfSubStr\pgph@dx\@backslashchar{}{%
572             \global\let\pgph@cbaseg\pgph@x}%
573         \fi
574     \endgroup
575     \ifx\pgph@cbaseg\relax\else\edef\pgph@cbase{[\pgph@cbaseg]}\fi
576     \fi\fi
577 }{}%
578 \edef\pgph@clabel{%
579     \ifcsname pgph@citenote@#1\endcsname
580     \csname pgph@citenote@#1\endcsname, \fi
581     \pgph@cbase}%
582 }

```

`\pgph@render` `autorun` must not re-render a graph that has not changed. Rendering unconditionally is not merely wasteful: the image Graphviz produces is not guaranteed to be reproducible (its `cairo`-based outputs, `pdf` among them, carry a creation date), so an unconditional run makes the embedded image differ at every compilation. A build tool such as `latexmk` then sees an input file that never settles and reruns until it gives up, reporting that too many passes were needed.

We therefore render only when something relevant has changed. The signature of a rendering is the checksum of the `.dot` file just written, together with the engine and the format, which the `.dot` does not record; it is stored in the `.aux` by `\pgph@lastrender` and compared with the signature of the next run. Graphviz is run when the signature differs, when no signature is known yet (first run), when either output file is missing, and, as a safe fallback, when the engine provides no file checksum.

Both signatures are `\detokenized` before being compared. The checksum comes out of `\pdf@filemdfivesum` as characters of category *other*, while the same string read back from the `.aux` has its letters as category *letter*, so an `\ifx` on the raw strings would never see two runs as equal.

```

583 \let\pgph@prevsig\relax
584 \newcommand\pgph@lastrender[1]{\xdef\pgph@prevsig{\detokenize{#1}}}
585 \newcommand\pgph@render{%
586     \immediate\write18{\pgph@engine\space -T\pgph@format\space
587         \pgph@file.dot -o \pgph@file-proofgraph.\pgph@format}%
588     \immediate\write18{\pgph@engine\space -Tplain\space
589         \pgph@file.dot -o \pgph@file-proofgraph.plain}%
590 }
591 \newcommand\pgph@maybrender{%
592     \edef\pgph@sig{\ifdefined\pdf@filemdfivesum
593         \pdf@filemdfivesum{\pgph@file.dot}\fi}%
594     \ifx\pgph@sig\@empty
595         \pgph@render
596     \else
597         \edef\pgph@sig{\pgph@sig-\pgph@engine-\pgph@format}%
598         \edef\pgph@sig{\expandafter\detokenize\expandafter{\pgph@sig}}%
599         \IfFileExists{\pgph@file-proofgraph.\pgph@format}%

```

```

600     {\IfFileExists{\pgph@file-proofgraph.plain}%
601      {\ifx\pgph@sig\pgph@prevsig\else\pgph@render\fi}%
602      {\pgph@render}}}%
603     {\pgph@render}%

```

The signature is recorded whether or not we have just rendered: either way the output files on disk are the ones this `.dot` produces. The write must be `\immediate`, as `\pgph@emit` runs at end of document, where a deferred write would have no shipout left to carry it.

```

604     \if@filesw
605     \immediate\write\@auxout{\string\pgph@lastrender{\pgph@sig}}}%
606     \fi
607 \fi
608 }

```

`\pgph@emit` Open the file, resolve editing rules, write nodes then edges, close, and optionally run Graphviz.

```

609 \newwrite\pgph@out
610 \newwrite\pgph@mapout
611 \newcommand\pgph@emit{%
612   \immediate\openout\pgph@out=\pgph@file.dot\relax
613   \immediate\openout\pgph@mapout=\pgph@file-proofgraph.map\relax
614   \immediate\write\pgph@out{digraph proofgraph \pgph@ob}%
615   \immediate\write\pgph@out{\space\space rankdir="\pgph@rankdir";}%
616   \begingroup
617     \pgph@ignores
618     \pgph@excludes
619     \pgph@nodes
620     \pgph@edges
621   \endgroup
622   \immediate\write\pgph@out{\pgph@cb}%
623   \immediate\closeout\pgph@out
624   \immediate\closeout\pgph@mapout
625   \ifpgph@autorun\pgph@maybrender\fi
626 }

```

The emission is registered at `\begin{document}` rather than at load time so that it is added to the end-document hook *after* packages loaded later (notably `apxproof`, whose appendix, with the deferred proofs and their references, is built at end of document). This way the graph is written only once that material has been typeset, and the emission does not disturb the verbatim replay of deferred proofs.

```

627 \AtBeginDocument{\AtEndDocument{\pgph@emit}}

```

If TikZ is available, load its `calc` library now (at `\begin{document}`, where category codes are normal) so that the hyperlink overlay in `\proofgraph` can use it without reading a file mid-document.

```

628 \AtBeginDocument{\@ifpackageloaded{tikz}{\usetikzlibrary{calc}}{}}

```

`\pgphnodemap` One entry of the node-to-target map written by `\pgph@emit`: it records, for a `\pgphnodemapcite` Graphviz node identifier, what the node should link to. `\pgphnodemap` records the user *label* of the result a node represents; `\pgphnodemapcite` records the $\text{\LaTeX}$ *key* of an external citation node, so that (when the cited work is in the same document) the node can link to its bibliography entry, `hyperref's cite.<key>`

destination. The map is read back when the graph is embedded, to turn each node into a hyperlink.

```
629 \newcommand\pgphnodemap[2]{\global\@namedef{pgph@lbl@#1}{#2}}
630 \newcommand\pgphnodemapcite[2]{\global\@namedef{pgph@cbl@#1}{#2}}
```

\proofgraph Include the rendered graph (from a previous run) in `autorun` mode. When `hyperref` and `TikZ` are both available (and `hyperlinks` is not switched off), each node is made into an internal hyperlink to what it represents: a result node links to the result (its `\label`), and an external citation node (from the `cite` option) links to the cited work's bibliography entry, `hyperref`'s `cite.<key>` destination, when that work is cited in the same document. `Graphviz` produces a `.pdf` whose link annotations would be lost by `\includegraphics` (they sit inside a form `XOBJECT`), so instead we overlay transparent link areas at the node positions, which we read from the `-Tplain` output. Without those packages we fall back to a plain inclusion.

```
631 \newif\ifpgph@canlink
632 \newif\ifpgph@link
633 \newsavebox\pgph@imgbox
634 \newsavebox\pgph@natbox
635 \newread\pgph@plainin
636 \def\pgph@stop{}
637 \newcommand\proofgraph[1][{}]{%
638   \@ifundefined{includegraphics}{%
639     {\PackageError{proofgraph}{\string\proofgraph\space requires the
640       graphicx package}{Add \string\usepackage{graphicx} to your preamble
641       to embed the rendered graph.}}}%
642     {\IfFileExists{\pgph@file-proofgraph.\pgph@format}%
643       {\pgph@includegraph{#1}}}%
644     {\PackageWarning{proofgraph}{Rendered graph
645       \pgph@file-proofgraph.\pgph@format\space not found; compile with
646       shell-escape and the autorun option, then re-run}}}%
647 }
648 \newcommand\pgph@includegraph[1]{%
649   \pgph@canlinkfalse
650   \ifpgph@hyperlinks
651     \ifpackageloaded{hyperref}%
652       {\@ifpackageloaded{tikz}%
653         {\IfFileExists{\pgph@file-proofgraph.plain}%
654           {\pgph@canlinktrue}{}}}{}}}%
655   \fi
656   \ifpgph@canlink
657     \pgph@graphlinked{#1}%
658   \else
659     \includegraphics[#1]{\pgph@file-proofgraph.\pgph@format}%
660   \fi
661 }
```

The linked version. We read the node-to-target map under safe category codes (a label may contain `_` or, under `babel`, an active `:`), measure the rendered image, place it in a `TikZ` node, then read the `-Tplain` file and drop one transparent link box on each mapped node. `Graphviz` draws the graph inset by a margin, so the image is larger than the layout bounding box the `-Tplain` coordinates live in; we recover that margin by also measuring the image at its *natural* size and comparing it with the `-Tplain` graph size (the `graph` line gives it in inches, hence

the factor 72). Node positions are then expressed as fractions of the natural image, so they line up with the drawn nodes whatever size the image is finally scaled to.

```

662 \newcommand\pgph@graphlinked[1]{%
663   \begin{group}
664     \catcode'\:=12 \catcode'\_ =12 \catcode'\;=12 \catcode'\!=12
665     \catcode'\?=12 \catcode'\&=12 \catcode'\^ =12 \catcode'\~ =12
666     \InputIfFileExists{\pgph@file-proofgraph.map}{\fi}%
667   \end{group}
668   \sbox\pgph@imgbox{%
669     \includegraphics[#1]{\pgph@file-proofgraph.\pgph@format}}%
670   \sbox\pgph@natbox{%
671     \includegraphics{\pgph@file-proofgraph.\pgph@format}}%
672   \edef\pgph@imgw{\the\wd\pgph@imgbox}%
673   \edef\pgph@imght{\the\dimexpr\ht\pgph@imgbox+\dp\pgph@imgbox\relax}%
674   \edef\pgph@natw{\the\wd\pgph@natbox}%
675   \edef\pgph@nath{\the\dimexpr\ht\pgph@natbox+\dp\pgph@natbox\relax}%
676   \begin{tikzpicture}
677     \node[inner sep=\z@,outer sep=\z@](pgph@P){\usebox\pgph@imgbox};
678     \openin\pgph@plainin=\pgph@file-proofgraph.plain\relax
679     \pgph@loopplain
680     \closein\pgph@plainin
681   \end{tikzpicture}%
682 }
683 \newcommand\pgph@loopplain{%
684   \unless\ifeof\pgph@plainin
685     \read\pgph@plainin to \pgph@line
686     \pgph@procline
687     \expandafter\pgph@loopplain
688   \fi
689 }
690 \newcommand\pgph@procline{%
691   \IfBeginWith{\pgph@line}{graph }%
692     {\expandafter\pgph@dograph\pgph@line\pgph@stop}%
693     {\IfBeginWith{\pgph@line}{node }%
694       {\expandafter\pgph@donode\pgph@line\pgph@stop}{}}%
695 }
696 \def\pgph@dograph graph #1 #2 #3 #4\pgph@stop{%
697   \def\pgph@gw{#2}\def\pgph@gh{#3}}
698 \def\pgph@donode node #1 #2 #3 #4 #5 #6\pgph@stop{%
699   \StrDel{#1}{"}[\pgph@nid]%
700   \pgph@linkfalse
701   \ifcsname pgph@lbl@\pgph@nid\endcsname
702     \edef\pgph@tgt{\csname pgph@lbl@\pgph@nid\endcsname}%
703     \edef\pgph@thislink{\noexpand\hyperref[\pgph@tgt]}%
704     \pgph@linktrue
705   \else
706     \ifcsname pgph@clbl@\pgph@nid\endcsname
707       \edef\pgph@tgt{\csname pgph@clbl@\pgph@nid\endcsname}%
708       \edef\pgph@thislink{\noexpand\hyperlink{cite.\pgph@tgt}}%
709       \pgph@linktrue
710   \fi
711   \fi
712   \ifpgph@link
713     % Per-side Graphviz margin: half the natural-minus-layout size.

```

```

714 \pgfmathsetmacro\pgph@mx{(\pgph@natw-\pgph@gw*72)/2}%
715 \pgfmathsetmacro\pgph@my{(\pgph@nath-\pgph@gh*72)/2}%
716 % Node centre as a fraction of the natural image.
717 \pgfmathsetmacro\pgph@fx{(\pgph@mx+#2*72)/\pgph@natw}%
718 \pgfmathsetmacro\pgph@fy{(\pgph@my+#3*72)/\pgph@nath}%
719 \pgfmathsetlengthmacro\pgph@bw{#4*72/\pgph@natw*\pgph@imgw*0.92}%
720 \pgfmathsetlengthmacro\pgph@bh{#5*72/\pgph@nath*\pgph@imgh*0.9}%
721 \node[anchor=center]
722   at ($(\pgph@P.south west)!\pgph@fx!(\pgph@P.south east)$)%
723   !\pgph@fy!(\pgph@P.north west)!\pgph@fx!(\pgph@P.north east)$)$)
724   {\pgph@thislink{\phantom{\rule{\pgph@bw}{\pgph@bh}}}};%
725 \fi
726 }

727 \end{package}

```

Index

Numbers written in *italic* refer to the page where the corresponding entry is described; numbers underlined refer to the code line of the definition; numbers in roman refer to the code lines where the entry is used.

Symbols		C	
\!	664	\catcode	33, 34, 436, 664, 665
\"	436	\closein	680
\&	665	\Cref	283
\:	664	\cref	282
\;	664	\cs	17
\?	665	\csundef	90, 97
\@M	526	D	
\@Opargbegintheorem	272, 273, 274	\DeclareBoolOption	21, 26, 28
\@auxout	55, 605	\DeclareStringOption	20, 22, 23, 24, 25, 27, 29
\@backslashchar	547, 571	\detokenize	291, 348, 545, 564, 568, 584, 598
\@begintheorem	140, 141, 143, 147	\dimexpr	673, 675
\@currentlabel	191, 205	\dp	673, 675
\@ifpackageloaded	5, 306, 393, 628, 651, 652	E	
\@ifundefined	6, 10, 535, 638	\end	681
\@namedef	46, 48, 62, 91, 98, 104, 188, 419, 428, 433, 453, 498, 507, 629, 630	\endproof	7, 8, 11
\@opargbegintheorem	142, 151, 152, 153, 261, 262, 263	\expandarg	441
\@protected@testopt	291	\ExplSyntaxOff	18
\@spbegintheorem	157, 158, 159	\ExplSyntaxOn	16
\@spopargbegintheorem	162, 163, 164, 266, 267, 268	H	
\[	33	\hbox	175, 285, 528
\]	436	\ht	673, 675
\{	34, 519	\hyper@@link	524
\}	34, 437	\hyperlink	708
\]	33	\hyperref	703
\^	665	I	
_	664	\if@filesw	604
\	436	\IfBeginWith	60, 475, 565, 691, 693
\~	665	\IfBooleanTF	235
A		\ifboolexpr	485
\apxproofhook	311	\ifdefstring	392, 486, 492, 555
\AtBeginDocument	124, 305, 627, 628	\ifeof	684
\AtBeginEnvironment	105, 294	\IfFileExists	599, 600, 642, 653
\AtEndDocument	627	\ifnumequal	485
\AtEndPreamble	390	\ifpgph@apxproof	304
\AtEveryCitekey	393	\ifpgph@autorun	625
\autoref	283	\ifpgph@canlink	631, 656
\axp@forward@link	184, 204	\ifpgph@cite	257, 316, 391
B		\ifpgph@hyperlinks	650
\begin	676	\ifpgph@link	632, 712
		\ifpgph@natnum	520, 531
		\IfSubStr	292, 547, 571

<code>\iftrue</code>	536	<code>\pgph@cap@cite@optii</code>	329, 332
<code>\ignorespaces</code>	525	<code>\pgph@cap@cite@x</code>	323, 324
<code>\includegraphics</code>	659, 669, 671	<code>\pgph@cap@citegen</code>	321, 378
<code>\InputIfFileExists</code>	666	<code>\pgph@cap@citerec</code> .	331, 333, 334, 335
J			
<code>\jobname</code>	31	<code>\pgph@cap@Cref</code>	227
L			
<code>\label</code>	136, 137	<code>\pgph@cap@ceref</code>	226
M			
<code>\mbox</code>	528	<code>\pgph@cap@eqref</code>	230
<code>\meaning</code>	290	<code>\pgph@cap@ref</code>	225
N			
<code>\newcount</code>	38, 39	<code>\pgph@cap@vref</code>	232
<code>\NewDocumentCommand</code>	233, 278	<code>\pgph@cap@zcref</code>	233
<code>\newif</code>	304, 520, 631, 632	<code>\pgph@cb</code>	36, 622
<code>\newread</code>	635	<code>\pgph@cbase</code>	554, 557, 575, 581
<code>\newsavebox</code>	633, 634	<code>\pgph@cbaseg</code>	560, 572, 575
<code>\newtheorem</code>	71, 72	<code>\pgph@citecmd</code>	322, 326
<code>\nobreakspace</code>	527	<code>\pgph@citecmds</code>	361
<code>\node</code>	677, 721	<code>\pgph@citeedge</code>	476, 502
<code>\noexpand</code>	133, 214, 326, 703, 708	<code>\pgph@citelabel</code>	392, 555
O			
<code>\openin</code>	678	<code>\pgph@citesanitize</code> .	345, 522, 541, 562
P			
<code>\PackageError</code>	639	<code>\pgph@citeslot</code>	339, 343
<code>\PackageWarning</code>	644	<code>\pgph@citeslotcnt</code>	39, 352, 353
<code>\pdf@filemdfivesum</code>	592, 593	<code>\pgph@citestyle</code>	420, 421, 512
<code>\penalty</code>	526	<code>\pgph@ckey</code> 553, 554, 556, 557, 559, 563	
<code>\pgfmathsetlengthmacro</code>	719, 720	<code>\pgph@clabel</code>	509, 512, 578
<code>\pgfmathsetmacro</code> ..	714, 715, 717, 718	<code>\pgph@class@endproof</code>	7, 11
<code>\pgph@adddge</code>	213, 222, 340	<code>\pgph@class@proof</code>	7, 11
<code>\pgph@apxinit</code>	307, 310	<code>\pgph@cone</code>	372, 373
<code>\pgph@apxproofcapture</code>	310	<code>\pgph@conename</code>	371, 372
<code>\pgph@apxprooftrue</code>	307	<code>\pgph@curproof</code> 45, 172, 173, 176, 219,	
<code>\pgph@auxmap</code>	46	222, 255, 313, 314, 336, 340, 399	
<code>\pgph@beginproof</code>	225, 294, 300	<code>\pgph@curresult</code>	44,
<code>\pgph@beginresult</code>	105, 126	131, 133, 170, 173, 181, 186,	
<code>\pgph@bh</code>	720, 724	188, 192, 199, 200, 201, 206, 255	
<code>\pgph@blxkey</code>	542, 548	<code>\pgph@curslot</code>	
<code>\pgph@blxrecord</code>	393, 539		340, 350, 353, 354, 355, 357
<code>\pgph@bschar</code>	442, 443	<code>\pgph@defaultenvs</code>	84, 125
<code>\pgph@bw</code>	719, 724	<code>\pgph@defaulttexclue</code>	87, 101
<code>\pgph@candidates</code> 57, 63, 64, 66, 119, 120		<code>\pgph@direction</code>	492
<code>\pgph@canlinkfalse</code>	649	<code>\pgph@dn</code>	463, 465
<code>\pgph@canlinktrue</code>	654	<code>\pgph@dograph</code>	692, 696
<code>\pgph@cap@autoref</code>	228	<code>\pgph@dohook</code>	102, 110, 112
<code>\pgph@cap@cite@noopt</code>	327, 334	<code>\pgph@donode</code>	694, 698
<code>\pgph@cap@cite@opt</code>	327, 328	<code>\pgph@dotesc</code>	435, 466, 509
<code>\pgph@cap@cite@opti</code>	329, 330	<code>\pgph@dqchar</code>	443
		<code>\pgph@dx</code> ..	545, 547, 564, 565, 568, 571
		<code>\pgph@edge</code>	214, 474
		<code>\pgph@edges</code>	41, 215, 403, 620
		<code>\pgph@ef</code>	406, 407, 410
		<code>\pgph@emit</code>	609
		<code>\pgph@emitedge</code>	487, 491, 516
		<code>\pgph@emitnode</code>	451, 456, 460
		<code>\pgph@engine</code>	586, 588, 597
		<code>\pgph@excluderule</code>	417, 431
		<code>\pgph@excludes</code>	43, 417, 618
		<code>\pgph@f</code>	426, 427, 428

<code>\pgph@file</code>	31, 587, 589, 593, 599, 600, 612, 613, 642, 645, 653, 659, 666, 669, 671, 678
<code>\pgph@format</code>	586, 587, 597, 599, 642, 645, 659, 669, 671
<code>\pgph@fx</code>	717, 722, 723
<code>\pgph@fy</code>	718, 723
<code>\pgph@gh</code>	697, 715
<code>\pgph@gnotestr</code>	346, 348, 356, 357
<code>\pgph@graphlinked</code>	657, 662
<code>\pgph@gw</code>	697, 714
<code>\pgph@gxdefcs</code>	319, 354, 355, 357, 548
<code>\pgph@hookcite</code>	319
<code>\pgph@hooknames</code>	126
<code>\pgph@hookproofhead</code>	258, 260
<code>\pgph@idcnt</code>	38, 130, 131
<code>\pgph@ignorerule</code>	414, 425
<code>\pgph@ignores</code>	42, 414, 617
<code>\pgph@imgbox</code>	633, 668, 672, 673, 677
<code>\pgph@imgh</code>	673, 720
<code>\pgph@imgw</code>	672, 719
<code>\pgph@includegraph</code>	643, 648
<code>\pgph@install</code>	237, 250, 251, 252
<code>\pgph@installcite</code>	257, 316, 361
<code>\pgph@installciteone</code>	373, 375
<code>\pgph@installrefs</code>	174, 249, 256, 315
<code>\pgph@k</code>	221, 222, 338, 339, 409, 410
<code>\pgph@key</code>	505, 514
<code>\pgph@labeledge</code>	403, 405
<code>\pgph@lastrender</code>	583
<code>\pgph@lbl</code>	465, 466, 468
<code>\pgph@line</code>	685, 691, 692, 693, 694
<code>\pgph@linkfalse</code>	700
<code>\pgph@linktrue</code>	704, 709
<code>\pgph@loopplain</code>	679, 683, 687
<code>\pgph@maplabel</code>	136, 137, 198
<code>\pgph@mapout</code>	470, 513, 610, 613, 624
<code>\pgph@maybehook</code>	108, 121
<code>\pgph@maybrender</code>	583, 625
<code>\pgph@mx</code>	714, 717
<code>\pgph@my</code>	715, 718
<code>\pgph@natbox</code>	634, 670, 674, 675
<code>\pgph@natdo</code>	532, 567
<code>\pgph@nath</code>	675, 715, 718, 720
<code>\pgph@natnumfalse</code>	534
<code>\pgph@natnumset</code>	533, 566
<code>\pgph@natnumtrue</code>	537
<code>\pgph@natpick</code>	530, 532
<code>\pgph@natw</code>	674, 714, 717, 719
<code>\pgph@nid</code>	699, 701, 702, 706, 707
<code>\pgph@nil</code>	521, 530, 532
<code>\pgph@nm</code>	447, 449, 465
<code>\pgph@node</code>	133, 445
<code>\pgph@nodes</code>	40, 127, 619
<code>\pgph@nt@plain</code>	72, 73
<code>\pgph@nt@star</code>	72, 75
<code>\pgph@ob</code>	35, 614
<code>\pgph@obrace</code>	519, 565
<code>\pgph@one</code>	89, 90, 91, 92, 96, 97, 98, 116, 117, 118, 120, 121, 220, 221, 337, 338, 408, 409
<code>\pgph@orig@bt</code>	141, 145, 148
<code>\pgph@orig@newtheorem</code>	71, 74, 75
<code>\pgph@orig@obt</code>	152, 155
<code>\pgph@orig@Obtp</code>	273, 275
<code>\pgph@orig@obtp</code>	262, 264
<code>\pgph@orig@spbt</code>	158, 160
<code>\pgph@orig@spnewtheorem</code>	77, 81, 82
<code>\pgph@orig@spobt</code>	163, 166
<code>\pgph@orig@spobtp</code>	267, 270
<code>\pgph@out</code>	467, 499, 510, 609, 612, 614, 615, 622, 623
<code>\pgph@plainedge</code>	410, 477, 479
<code>\pgph@plainin</code>	635, 678, 680, 684, 685
<code>\pgph@prependnode</code>	126, 134
<code>\pgph@prevsig</code>	583, 584, 601
<code>\pgph@procline</code>	686, 690
<code>\pgph@proofinner</code>	298, 301, 302
<code>\pgph@proofmeaning</code>	290, 292
<code>\pgph@proofopt</code>	301, 302
<code>\pgph@rankdir</code>	615
<code>\pgph@recordref</code>	218, 225, 226, 227, 229, 231, 232, 234, 396
<code>\pgph@recordresult</code>	126
<code>\pgph@redefproof</code>	293, 297
<code>\pgph@register</code>	57, 74, 81, 92, 118
<code>\pgph@render</code>	583
<code>\pgph@replay</code>	325, 331, 333, 334
<code>\pgph@resolve</code>	313, 406, 422, 426, 432, 480
<code>\pgph@resolveregistration</code>	115, 125
<code>\pgph@s</code>	461, 468
<code>\pgph@s@saved@autoref</code>	229
<code>\pgph@s@saved@Cref</code>	227
<code>\pgph@s@saved@cref</code>	226
<code>\pgph@s@saved@eqref</code>	231
<code>\pgph@s@saved@ref</code>	225
<code>\pgph@s@saved@vref</code>	232
<code>\pgph@s@saved@zceref</code>	235
<code>\pgph@s@saved@curproof</code>	172, 176
<code>\pgph@s@saved@label</code>	136, 211
<code>\pgph@s@scanproofarg</code>	264, 269, 275, 278
<code>\pgph@s@scanrefs</code>	144, 154, 165, 169
<code>\pgph@s@selfloops</code>	486
<code>\pgph@s@setcitelabel</code>	502
<code>\pgph@s@setmap</code>	49, 199
<code>\pgph@s@sig</code>	348, 349, 350, 354, 592, 594, 597, 598, 601, 605

capture no longer reinstalls over itself	1	over, made the skip overshoot and swallow the writes emitting the node, which Graphviz then drew from its edges alone: unstyled, labelled by its internal name and without a hyperlink. It affected every first run, and biblatex documents always	1
Support, with the cite option, the two-optional-argument citation form [pre] [post] { key } of natbib and biblatex ; it previously broke compilation with confusing errors inside proofs . .	1	Recover the citation tag of the numeric and alphabetic biblatex styles for citelabel=tag , which records no citation tag in the .aux for it to read	1
v1.0.1-dev		Require a LaTeX2e format of October 2020 or later, which the use of expl3 and NewDocumentCommand already implied	1
General: Capture zcref of zref-clever , in its starred, optional-argument and multiple-label forms, like the other cross-referencing commands	1	With autorun , run Graphviz only when the graph, the engine or the format has changed. Rendering at every compilation kept producing a different image, since Graphviz stamps a creation date into its cairo -based outputs, and latexmk would then rerun until it reported that too many passes were needed	1
Capture, with the cite option, every citation command of natbib and biblatex , such as textcite and parencite , down to those printing a mere fragment of a citation such as citeauthor , and add proofgraphcitemcommand to capture further ones	1		
Emit citation nodes again when citelabel=tag finds no tag to use. A stray iftrue , counted as a conditional while a false conditional was being skipped			